```
type StarCount = 0 | 1 | 2 | 3 | 4
```

The meaning is the same as string literal types. TypeScript doesn't have a range type, but a numeric literal can be used if your range is finite and made of integers.

One thing worth mentioning is that, for all purposes other than type checking, the literal type behaves like a string or number. This means you can assign our `TicketStatus` to a string or add a `StarCount` to another number. But the result of say, `StarCount + number` is a `number`, not a `StarCount`. In other words, you can perform operations on your literal types, but the result will be typed as whatever the underlying type is, not the literal type.

Now we get to what we actually used to build the actions in our reducer, which is a TypeScript concept called a *discriminated union*. A discriminated union takes a set of interfaces that have a common property that is defined with a different string literal (the discriminator) and a union type that combines them (the union).

In our reducer, we have a bunch of action interfaces that have a `type` property with a string value, like these:

chapter_13/01/app/javascript/contexts/venue_types.ts

```
interface HoldTicket {
  type: "holdTicket"
  seatNumber: number
  rowNumber: number
}
```

And a union type that combines all the actions:

chapter_13/01/app/javascript/contexts/venue_types.ts

```
export type VenueAction =
  | ClearHolds
  | HoldTicket
  | InitFromProps
  | SetTicketToBuy
```